

KAÇAN NESNE TAKİP OYUNU

E M U 8 0 8 6

BLM2008 MİKORİŞLEMCİLER DERSİ

170424841 - E s m a S u d e K a r a d a ğ

EMU8086 İLE KAÇAN NESNE TAKİBİ OYUNU

Bu projede 8086 Assembly dili kullanılarak basit bir oyun geliştirilmişti. Oyunumda ekranda hareket eden kırmızı bir nesne bulunur. Kullanıcı mouse ile bu nesneye tıklamaya çalışır. Her başarılı tıklamada skor artar, nesne küçülür ve oyun zorlaşır.

Bu projede gerçekleştirdiklerim:

- 8086 Assembly dili ile etkileşimli bir oyun geliştirmek
- Timer tabanlı hareket sistemi oluşturmak
- Mouse ile tıklama kontrolü yapmak
- Skor ve süre bilgisini ekranda göstermek
- Nesnenin her tıklamada küçülmesini ve hızlanmasını sağlamak
- BIOS ve DOS interruptlarını kullanmak

PROGRAMIN GENEL YAPISI

```
new open examples save compile emul
001 #make_COM#
002
003 ORG 100h
004
005 JMP BASLA
006
007 I
```

Değişkenler Ve Oyun Verileri:

```
014 nesne_x DW 35
015 nesne_y DW 10
016 nesne_boy DW 4
017 nesne_dx DW 1
018 nesne_dy DW 1
019 skor DW 0
020 sure DW 30
021 son_san DW 0
022 son_har DW 0
023 oyun_dur DB 0
024 har_hz DW 3
025 tohum DW 0
026 hiz_say DW 0
027 fare_var DB 0
028 tik_now DW 0
```

- ORG 100h, programın COM dosyası olarak çalışacağını gösterir.
- COM programlarında kod bellekte 100h adresinden başlar.
- JMP BASLA, değişkenlerin üzerinden atlayarak doğrudan ana programa geçilmesini sağlar.

Değişken

nesne_x

nesne_y

nesne_boy

nesne_dx

nesne_dy

skor

sure

oyun_dur

har_hz

fare_var

Görevi

Nesnenin yatay konumu

Nesnenin dikey konumu

Nesnenin boyutu

X yönündeki hareket

Y yönündeki hareket

Yakalama sayısı

Oyun süresi

Oyunun bitip bitmediği

Nesnenin hareket hızı

Mouse var mı kontrolü

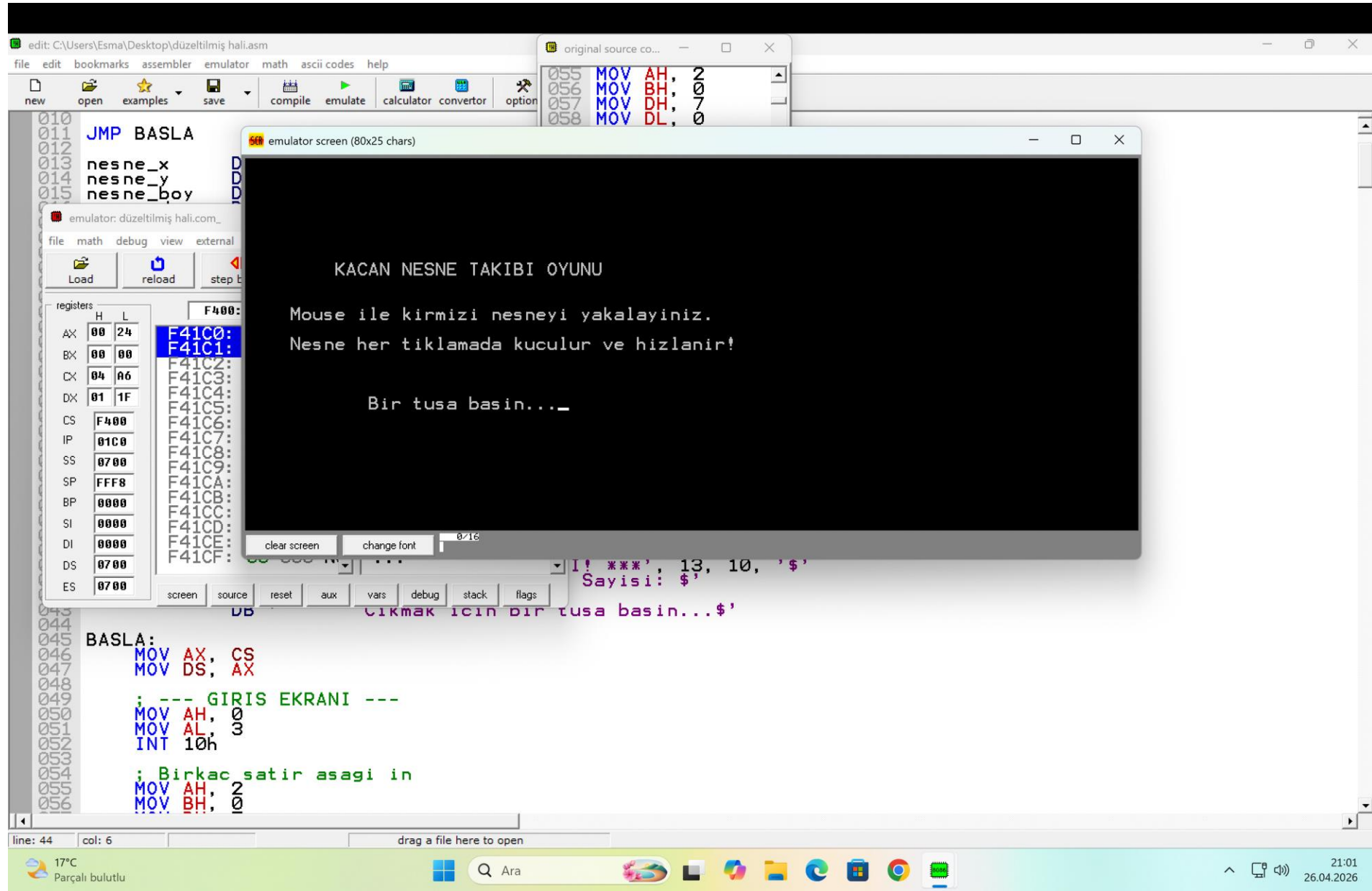
EKRAN MESAJLARI

```

msg_base DB '          KACAN NESNE TAKIBI OYUNU', 13, 10
          DB 13, 10, 13, 10
          DB '          Mouse ile kirmizi nesneyi yakalayınız.', 13, 10
          DB 13, 10
          DB '          Nesne her tıklamada kuculur ve hızlanır!', 13, 10
          DB 13, 10, 13, 10, 13, 10
          DB '          Bir tusa basın...', '$'
msg_sk   DB 'Skor: $'
msg_sr   DB 'Sure: $'
msg_bo   DB '$'
msg_bt   DB '          *** OYUN BITTI! ***', 13, 10, '$'
msg_tp   DB '          Toplam Yakalama Sayisi: $'
msg_ck   DB 13, 10, 13, 10
          DB '          Cikmak icin bir tusa basin...$'

```

- Oyunda kullanılacak yazılar bellekte saklanır.
- DOS INT 21h / AH=09h fonksiyonu \$ karakterine kadar olan metni ekrana yazar.
- 13, 10 değerleri satır başı ve alt satıra geçme için kullanılır.



VERİ SEGMENTİNİN AYARLANMASI

```

041 |
042 | BASLA:
043 |     MOV AX, CS
044 |     MOV DS, AX
045 |

```

- CS, kod segmentini gösterir.
- DS, veri segmentini gösterir.
- COM programlarında kod ve veri aynı segmentte bulunur.
- Bu yüzden CS değeri DS içine kopyalanır.

Ekranı Text Moduna Alma:

```

045 |
046 | ;GIRIS EKRANI
047 | MOV AH, 0
048 | MOV AL, 3
049 | INT 10h
050 |

```

- INT 10h, BIOS video servisi.
- AH=00h, video modunu değiştirme fonksiyonudur.
- AL=03h, 80x25 text modudur.
- Program ekranı temizleyip standart yazı moduna alır.

Bu oyun grafik modunda değil, text modunda çalışıyor. INT 10h ile ekran 80 sütun ve 25 satırdan oluşan klasik text moda alınmıştır. Nesne aslında karakterlerden oluşan kırmızı bir bloktur.

GİRİŞ EKRANI VE KLAVYE BEKLEME

```
058 MOV DX, OFFSET msg_bas
059 MOV AH, 9
060 INT 21h
061
062 ;Tus bekle
063 MOV AH, 0
064 INT 16h
```

- INT 21h / AH=09h, ekrana yazı yazdırır.
- DX, yazdırılacak mesajın adresini tutar.
- INT 16h / AH=00h, klavyeden tuş bekler.

Önce giriş mesajı ekrana yazdırılıyor. Daha sonra INT 16h ile kullanıcıdan bir tuş bekleniyor. Kodumda basılan tuştan gelen değer **tohum** değişkenine aktarılmıştır. Böylece rastgele sayı üretimi için başlangıç değeri oyunucunun girişinden almış oldum.

MOUSE KONTROLÜNÜN BAŞLATILMASI

```

0078      MOV     AX, 0
0079      INT     33h
0080      CMP     AX, 0
0081      JE      FARE_YOK
0082      MOV     AL, 1
0083      MOV     [fare_var], AL
0084      MOV     AX, 1
0085      INT     33h
0086      JMP     FARE_OK
0087 FARE_YOK:
0088      MOV     AL, 0
0089      MOV     [fare_var], AL
0090 FARE_OK:
0091

```

- INT 33h, mouse servisi.
- AX=0, mouse sürücüsünü kontrol eder.
- Eğer AX=0 dönerse mouse yoktur.
- Mouse varsa fare_var=1 yapılır.
- AX=1 ile mouse imleci gösterilir.

Bu oyunda oyuncu mouse ile nesneyi yakaladığı için önce mouse var mı diye kontrol yapan kısmı ekledim. Sonra mouse varsa mouse'u aktif hale getiren kısım çalışıyor.

TİMER BAŞLANGICI

```
099  
100      MOV AH, 0  
101      INT 1Ah  
102      MOV [son_san], DX  
103      MOV [son_har], DX  
104      ADD [tohum], DX  
105
```

- INT 1Ah, BIOS zaman servsidir.
- AH=00h, sistem saatini okur.
- Dönen tick değeri CX:DX içinde tutulur.
- Burada DX değeri süre, hareket ve rastgele sayı için kullanılır.

Bu bölümde BIOS timer servisi olan **INT 1Ah** kullanılarak sistem tick değeri okunur. Okunan **DX** değeri hem saniye takibi hem de hareket zamanlaması için başlangıç değeri olarak alınır. Ayrıca bu değer **tohum** değişkenine eklenerek rastgele sayı üretimi daha değişken hale getirilir. Böylece oyun her çalıştığında nesnenin başlangıç konumu daha farklı olabilir.

RASTGELE BAŞLANGIÇ KONUMU

```
107 CALL RASTGELE
108 MOV DX, 0
109 MOV BX, 65
110 DIV BX
111 ADD DX, 3
112 MOV [nesne_x], DX
113
114 CALL RASTGELE
115 MOV DX, 0
116 MOV BX, 16
117 DIV BX
118 ADD DX, 2
119 MOV [nesne_y], DX
120
```

Kodumda nesne sabit bir noktadan başlamak yerine rastgele bir başlangıç konumuna yerleştirdim. RASTGELE fonksiyonu çağrılarak bir sayı ürettim. Daha sonra DIV komutu ile bu sayı belirli bir aralığa düşürüldü. Bölme işleminden kalan değer DX registerında tutulduğu için yeni X ve Y koordinatları bu değer üzerinden belirlenir.

ANA OYUN DÖNGÜSÜ

```

109
110 DONGU:
111     MOV AL, [oyun_dur]
112     CMP AL, 1
113     JE SONUC_EKRANI
114
115     MOV AH, 0
116     INT 1Ah
117     MOV [tik_now], DX
118
119     CALL ZAMAN_GUNCELLE
120
121     CALL HAREKET_KONTROL
122
123     |
124     MOV AL, [fare_var]
125     CMP AL, 1
126     JNE DONGU
127     CALL FARE_KONTROL
128
129     JMP DONGU
130
131
132
133

```

Bu bölüm oyunun ana döngüsüdür.
Her döngüde:

1. Oyun bitti mi kontrol edilir.
2. Güncel timer değeri alınır.
3. Süre güncellenir.
4. Nesnenin hareket edip etmeyeceği kontrol edilir.
5. Mouse tıklaması kontrol edilir.
6. Döngü tekrar başa döner.

SÜRE GÜNCELLEME KISMI

```

138 L
139 ZAMAN_GUNCELLE:
140     PUSH AX
141     PUSH BX
142     PUSH DX
143
144     MOV AX, [tik_now]
145     SUB AX, [son_san]
146     CMP AX, 18
147     JB ZG_SON
148
149     ; 1 saniye gecti
150     MOV AX, [tik_now]
151     MOV [son_san], AX
152
153     MOV AX, [sure]
154     DEC AX
155     MOV [sure], AX
156     CMP AX, 0
157     JG ZG_HIZ
158
159
160     MOV AL, 1
161     MOV [oyun_dur], AL
162     JMP ZG_SON
163

```

Süre güncelleme bölümünde güncel timer değeri ile son kaydedilen zaman değeri karşılaştırılır. Eğer aradaki fark 18 tick değerine ulaşmışsa yaklaşık 1 saniye geçmiş kabul edilir. Bu durumda sure değişkeni 1 azaltılır. Bu yöntem, işlemciyi boş yere bekletmeden zaman takibi yapmamızı sağlar. Yani süreyi klasik gecikme döngüsüyle değil, timer tabanlı olarak azalttırdım.

Bu kod blounda kalan sürenin sıfıra ulaşip ulaşmadığı kontrol etti. Eğer süre hâlâ sıfırdan büyükse oyun devam eder ama süre 0 olduğunda oyun_dur değişkeni 1 yapılır. Bu değişken, ana döngüde kontrol edildiği için oyun doğrudan sonuç ekranına yönlendirilir.

NESNENİN ZAMANLA HIZLANMASI

```

164 ZG_HIZ:
165     ; Her 5 saniyede hızlan
166     MOV AX, [hiz_say]
167     INC AX
168     MOV [hiz_say], AX
169     MOV DX, 0
170     MOV BX, 5
171     DIV BX
172     CMP DX, 0
173     JNE ZG_SON
174     MOV AX, [har_hz]
175     CMP AX, 1
176     JBE ZG_SON
177     DEC AX
178     MOV [har_hz], AX
179

```

- hiz_say her saniye artırılır.
- 5'e bölündüğünde kalan 0 ise 5 saniye geçmiş kabul edilir.
- har_hz azaltılır.
- har_hz azaldıkça nesne daha sık hareket eder.
- Minimum hız değeri 1 olarak korunur.

har_hz değişkeni nesnenin kaç tick aralıkla yeni konuma taşınacağını belirler. Bu değer azaldıkça nesne daha kısa aralıklarla yer değiştirir. Programda her 5 saniyede bir **har_hz** değeri azaltılarak oyunun zorluk seviyesi artırılır.

HAREKET KONTROLÜ

```

191 HAREKET_KONTROL:
192     PUSH AX
193
194     MOV AX, [tik_now]
195     SUB AX, [son_har]
196     CMP AX, [har_hz]
197     JB HK_SON
198
199     ; Hareket zamanı
200     MOV AX, [tik_now]
201     MOV [son_har], AX
202
203     CALL NESNE_SIL
204     CALL NESNE_HAREKET
205     CALL NESNE_CIZ
206     CALL BILGI_GOSTER
207

```

Nesnenin hareket zamanı gelip gelmediği kontrol edilir.

Eğer yeterli tick geçtiyse:

1. Eski nesne silinir.
2. Yeni koordinat hesaplanır.
3. Nesne tekrar çizilir.
4. Skor ve süre güncellenir.

Kod bloğu hareket zamanı geldiğinde önce nesnenin eski konumu siler. Daha sonra NESNE_HAREKET alt programı çağrılarak nesne yeni rastgele koordinatlara taşınır. Ardından nesne yeni konumunda tekrar çizilir ve skor-süre bilgileri güncellenir.

NESNEYİ SİLME

```

235     MOV     BH, 0
236     INT     10h
237
238     MOV     AH, 9
239     MOV     AL, 32
240     MOV     BL, 07h
241     MOV     BH, 0
242     MOV     CX, [nesne_boy]
243     INT     10h
244
245     INC     SI
246     JMP     NS_S

```

- Önce imleç nesnenin bulunduğu konuma alınır.
- AL=32, boşluk karakteridir.
- Nesnenin bulunduğu yere boşluk yazılarak eski görüntü silinir.
- CX, kaç tane karakter yazılacağını belirler.

NESNEYİ ÇİZME

```

279     INT     10h
280
281     MOV     AH, 9
282     MOV     AL, 219
283     MOV     BL, 0Ch
284     MOV     BH, 0
285     MOV     CX, [nesne_boy]
286     INT     10h
287
288     INC     SI
289     JMP     NC_S
290

```

- AL=219, dolu blok karakteridir.
- BL=0Ch, kırmızı renk bilgisidir.
- CX=nesne_boy, kaç blok çizileceğini belirler.
- Nesne kare şeklinde satır satır çizilir.

NESNENİN X VE Y HAREKETİ

NESNE_HAREKET :

```

322  PUSH  AX
323  PUSH  BX
324  PUSH  DX
325
326
327  CALL  RASTGELE
328  MOV   DX, 0
329  MOV   BX, 65
330  DIV   BX
331  ADD   DX, 3
332  MOV   [nesne_x], DX
333
334  CALL  RASTGELE
335  MOV   DX, 0
336  MOV   BX, 16
337  DIV   BX
338  ADD   DX, 2
339  MOV   [nesne_y], DX
340

```

Kodumda nesne, NESNE_HAREKET fonksiyonu çağırıldığında nesne için yeni rastgele X ve Y koordinatları üretilir. Böylece nesne farklı koulmlarda tekrardan benilirir.

Rastgele Konum Üretimi: Nesnenin ekran dışına çıkmaması için rastgele koordinatlar belirli aralıklarla sınırlandırdım. X koordinatı için DIV BX işleminde 65 değeri kullanılır ve sonuca 3 eklenir. Böylece X değeri yaklaşık 3 ile 67 arasında kalır. Y koordinatı için ise 16 değeri kullanılır ve sonuca 2 eklenerek nesnenin ekranda görünür bölgede kalması sağlanır.

RASTGELE SAYI ÜRETİMİ VE NESNENİN HAREKETİ

```
378  
379 RASTGELE:  
380 PUSH BX  
381 PUSH DX  
382 MOV AX, [tohum]  
383 MOV BX, 25173  
384 MUL BX  
385 ADD AX, 13849  
386 MOV [tohum], AX  
387 POP DX  
388 POP BX  
389 RET  
390
```

Bu kod bloğunda nesnenin rastgele konumlara taşınması için basit bir pseudo-random sayı üretici kullandım. Kod **tohum** değerini önce alınır, belirli bir sayı ile çarpar ve üzerine sabit bir değer ekler. Elde edilen değeri tekrar **tohum** değişkenine kaydeder. Bu fonksiyon sayesinde nesnenin başlangıç konumu ve oyun içindeki yeni konumları rastgeleye yakın şekilde belirlenir.

- tohum: rastgele sayı üretiminin başlangıç değeri
- MUL: çarpma işlemi yapar
- ADD: sabit değer ekler
- Yeni değer tekrar belleğe kaydedilir

MOUSE KONUMUNU OKUMA

```
402
403
404
405
```

```
MOV AX, 3
INT 33h
```

Sol tık kontrolü:

```
410
411
412
413
```

```
AND BX, 1
CMP BX, 1
JNE FK_E
```

Text Koordinatını Piksele Çevirme:

```
415
416
417
418
419
420
421
```

```
└ MOV AX, [nesne_x]
  ADD AX, AX
  ADD AX, AX
  ADD AX, AX
  CMP CX, AX
  JL FK_E
```

Bu bölümde INT 33h mouse interruptı kullanılarak mouse'un konumu ve buton durumu okuttum. CX mouse'un X koordinatını, DX Y koordinatını, BX ise buton bilgisini tutar. AND BX, 1 işlemi ile sol mouse tuşuna basılıp basılmadığı kontrol edilir.

- Text modunda her karakter yaklaşık 8 piksel genişliğindedir.
- Nesnenin X koordinatı 8 ile çarpılır.
- Çarpma işlemi ADD AX, AX komutu üç kez kullanılarak yapılır. Çünkü:
 1. ADD → 2 kat
 2. ADD → 4 kat
 3. ADD → 8 kat
- Mouse X konumu bu sınırla karşılaştırılır.

TIKLAMA ALANI KONTROLÜ

```

414 ; --- Sol sinir: nesne_x * 8 ---
415 MOV AX, [nesne_x]
416 ADD AX, AX
417 ADD AX, AX
418 ADD AX, AX
419 CMP CX, AX
420 JL FK_E
421
422 ; --- Sağ sinir: (nesne_x + nesne_boy) * 8 ---
423 MOV AX, [nesne_x]
424 ADD AX, [nesne_boy]
425 ADD AX, AX
426 ADD AX, AX
427 ADD AX, AX
428 CMP CX, AX
429 JGE FK_E
430
431 ; --- Üst sinir: nesne_y * 8 ---
432 MOV AX, [nesne_y]
433 ADD AX, AX
434 ADD AX, AX
435 ADD AX, AX
436 CMP DX, AX
437 JL FK_E
438
439 ; --- Alt sinir: (nesne_y + nesne_boy) * 8 ---
440 MOV AX, [nesne_y]
441 ADD AX, [nesne_boy]
442 ADD AX, AX
443 ADD AX, AX
444 ADD AX, AX
445 CMP DX, AX
446 JGE FK_E
447

```

Mouse koordinatları piksel cinsinden gelir, ancak nesne text modundaki satır-sütun koordinatlarıyla tutulur. Bu yüzden nesne koordinatı 8 ile çarpılarak piksel karşılığına çevrilir.

Mouse'un X ve Y konumu nesnenin sınırları içinde kalıyorsa tıklama başarılı kabul edilir.

BAŞARILI TIKLAMA VE SKOR ARTIŞI

```

449 MOV AX, [skor]
450 INC AX
451 MOV [skor], AX
452
453 CALL NESNE_SIL
454

```

- Mouse nesneye tıkladıysa skor artırılır.
- INC AX, AX değerini 1 artırır.
- Yeni skor tekrar belleğe yazılır.
- Eski nesne ekrandan silinir.

Nesneyi Her Tıklamada Küçültme:

```

457 MOV AX, [nesne_boy]
458 MOV BX, 9
459 MUL BX
460 MOV BX, 10
461 MOV DX, 0
462 DIV BX
463 CMP AX, 1
464 JGE FK_BOK
465 MOV AX, 1
466 FK_BOK:
467 MOV [nesne_boy], AX
468

```

- Nesne boyutu 9 ile çarpılır.
- Sonra 10'a bölünür.
- Bu işlem yaklaşık %10 küçültme anlamına gelir.
- Boyutun 1'den küçük olması engellenir.

SKOR VE SÜREYİ EKRANDA GÖSTERME

```

512 ; Skor
513 MOV AH, 2
514 MOV BH, 0
515 MOV DH, 0
516 MOV DL, 2
517 INT 10h
518 MOV DX, OFFSET msg_sk
519 MOV AH, 9
520 INT 21h
521 MOV AX, [skor]
522 CALL SAYI_YAZ
523 MOV DX, OFFSET msg_bo
524 MOV AH, 9
525 INT 21h
526
527 ; Sure
528 MOV AH, 2
529 MOV BH, 0
530 MOV DH, 0
531 MOV DL, 65
532 INT 10h
533 MOV DX, OFFSET msg_sr
534 MOV AH, 9
535 INT 21h
536 MOV AX, [sure]
537 CALL SAYI_YAZ
538 MOV DX, OFFSET msg_bo
539 MOV AH, 9
540 INT 21h

```

- İmleç ekranın üst satırına taşınır.
 - "Skor:" yazısı yazdırılır.
 - Skor değeri AX içine alınır.
 - SAYI_YAZ fonksiyonu çağrılarak sayı ekrana yazdırılır.
-
- İmleç ekranın üst satırında sağ tarafa taşınır.
 - DH = 0 satır numarasını belirtir.
 - DL = 65 sütun numarasını belirtir.
 - msg_sr içindeki "Sure:" yazısı ekrana yazdırılır.
 - sure değişkenindeki kalan süre AX registerına alınır.
 - SAYI_YAZ fonksiyonu çağrılarak süre ekrana sayı olarak yazdırılır.
 - msg_bo ile eski sayı kalıntılarını temizlemek için boşluk yazdırılır.

SAYI YAZDIRMA FONKSİYONU VE STACK

```

608 |
609 | SAYI_YAZ:
610 |     PUSH AX
611 |     PUSH BX
612 |     PUSH CX
613 |     PUSH DX
614 |
615 |     MOV CX, 0
616 |     MOV BX, 10
617 | SY_B:
618 |     MOV DX, 0
619 |     DIV BX
620 |     PUSH DX
621 |     INC CX
622 |     CMP AX, 0
623 |     JNE SY_B
624 |
625 | SY_Y:
626 |     POP DX
627 |     ADD DL, 30h
628 |     MOV AH, 2
629 |     INT 21h
630 |     LOOP SY_Y
631 |
632 |     POP DX
633 |     POP CX
634 |     POP BX
635 |     POP AX
636 |     RET

```

- Sayı 10'a bölünerek basamaklarına ayrılır.
- Kalan değerler DX içinde oluşur.
- Basamaklar stack'e atılır.
- CX, kaç basamak olduğunu tutar.
- Stack'ten basamaklar geri alınır.
- `ADD DL, 30h`, sayıyı ASCII karakterine çevirir.
- `INT 21h / AH=02h`, tek karakter yazdırır.
- `LOOP`, CX sıfır olana kadar döngüyü sürdürür.

SONUÇ EKRANI VE PROGRAMI

SONLANDIRMA

```

549
550 SONUC_EKRANI:
551     MOV AX, 2
552     INT 33h
553
554     MOV AH, 0
555     MOV AL, 3
556     INT 10h
557
558     MOV DX, OFFSET msg_bt
559     MOV AH, 9
560     INT 21h
561
562     MOV AX, [skor]
563     CALL SAYI_YAZI
564
565
566     MOV AH, 0
567     INT 16h
568
569     MOV AH, 4Ch
570     INT 21h
571

```

- Mouse imleci gizlenir.
 - Ekran temizlenir.
 - Oyun bitti mesajı yazdırılır.
 - Toplam skor ekrana basılır.
-
- Önce kullanıcıdan bir tuş beklenir.
 - INT 21h / AH=4Ch, programı DOS'a geri döndürür.
 - Program düzgün şekilde sonlandırılır.

Program Akış Şeması

